

Informe Tarea 1

Programación modular y medición de tiempos

Documento consolidado: Parte 1 y Parte 2

Lenguaje C - Desarrollo de código, diagramas de flujo y análisis de mediciones

El informe presenta la transformación de una solución monolítica a modular y la medición de tiempos de ejecución para distintos ciclos for.

Índice

1. Introducción
2. Objetivos de la tarea
3. Parte 1: Programación modular
 - 3.1 Código original
 - 3.2 Implementación modular y uso de enum
 - 3.3 Explicación y diagrama de flujo
4. Parte 2: Medición de tiempos de ejecución
 - 4.1 Hito 1: cicloEjemplo()
 - 4.2 Hito 2: cicloPrueba()
 - 4.3 Hito 3: resultados y análisis
5. Conclusiones

1. Introducción

La presente tarea aborda dos contenidos fundamentales del lenguaje C. En la Parte 1 se trabaja la programación modular a partir de una solución monolítica que manipula un arreglo de sensores. En la Parte 2 se realizan mediciones de tiempo de ejecución para comparar el costo computacional de distintos ciclos for, diferenciando entre un ciclo vacío y ciclos que realizan operaciones de Entrada/Salida.

2. Objetivos de la tarea

- Implementar código modular a partir de una solución monolítica.
- Utilizar tipos de datos enum para mejorar la legibilidad del código.
- Realizar mediciones de tiempo de ejecución para comparar operaciones en memoria y operaciones de Entrada/Salida.

3. Parte 1: Programación modular

3.1. Descripción del código original

El código inicial se encuentra escrito completamente dentro de la función `main()`. En él se declara un arreglo de sensores, se inicializan sus posiciones en cero, se leen valores simulados mediante `lecturaSensor()` y, finalmente, se muestran los resultados por pantalla. La lógica es correcta, pero la solución es monolítica porque todas las operaciones se concentran en una sola función.

```
int main(int argc, char* argv[]) {
    int sensores[NUM_SENORES];

    for (int i = 0; i < NUM_SENORES; i++) {
        sensores[i] = 0;
    }

    for (int i = 0; i < NUM_SENORES; i++) {
        sensores[i] = lecturaSensor();
    }
}
```

```

}

for (int i = 0; i < NUM_SENSORES; i++) {
    printf("Sensor %i: %i [mV]\n", i, sensores[i]);
}

exit(EXIT_SUCCESS);
}

```

3.2. Implementación modular y uso de enum

Para transformar la solución anterior en una versión modular, se implementa la función `usarSensores(sensores, acc)`. La variable `acc` indica la acción que se debe ejecutar, utilizando nombres claros mediante `enum`. Así, `main()` queda reducido a tres llamadas ordenadas.

```

typedef enum {
    INIT,
    READ,
    SHOW
} AccionSensor;

int main(int argc, char* argv[]) {
    int sensores[NUM_SENSORES];

    usarSensores(sensores, INIT);
    usarSensores(sensores, READ);
    usarSensores(sensores, SHOW);

    exit(EXIT_SUCCESS);
}

```

```

void usarSensores(int sensores[], AccionSensor acc) {
    if (acc == INIT) {
        for (int i = 0; i < NUM_SENSORES; i++) sensores[i] = 0;
    }
    if (acc == READ) {
        for (int i = 0; i < NUM_SENSORES; i++) sensores[i] = lecturaSensor();
    }
    if (acc == SHOW) {
        for (int i = 0; i < NUM_SENSORES; i++)
            printf("Sensor %i: %i [mV]\n", i, sensores[i]);
    }
}

```

3.3. Explicación y diagrama de flujo

El programa comienza declarando el arreglo de sensores. Luego, mediante la acción `INIT`, todas las posiciones se dejan en cero. Con `READ` se simula la lectura de cada sensor y con `SHOW` se imprimen los resultados. Esta reorganización mantiene el comportamiento del programa original, pero mejora el orden, la lectura y el mantenimiento del código.

Diagrama de flujo de la Parte 1

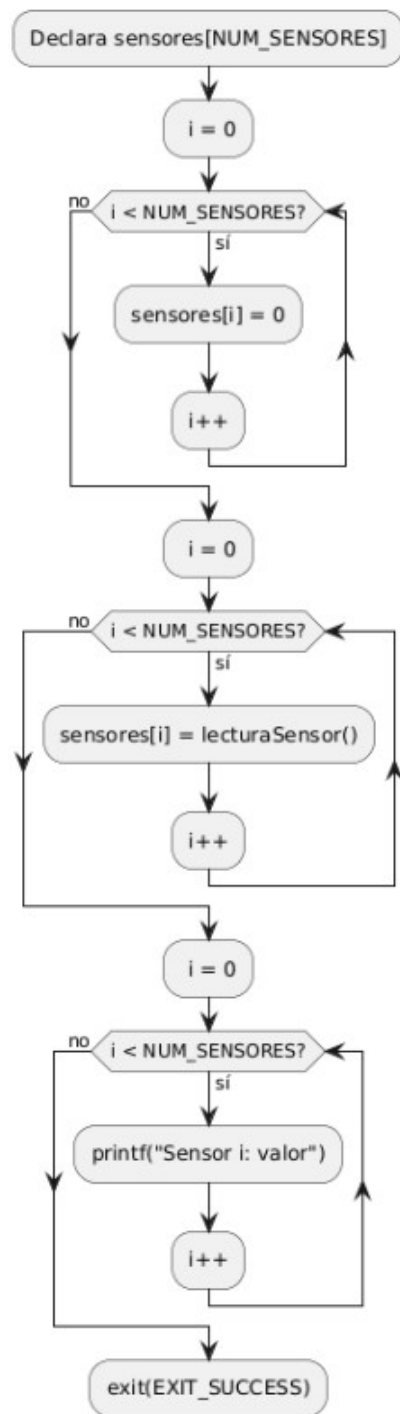


Figura 1. Diagrama de flujo de la solución original utilizada como base para la Parte 1.

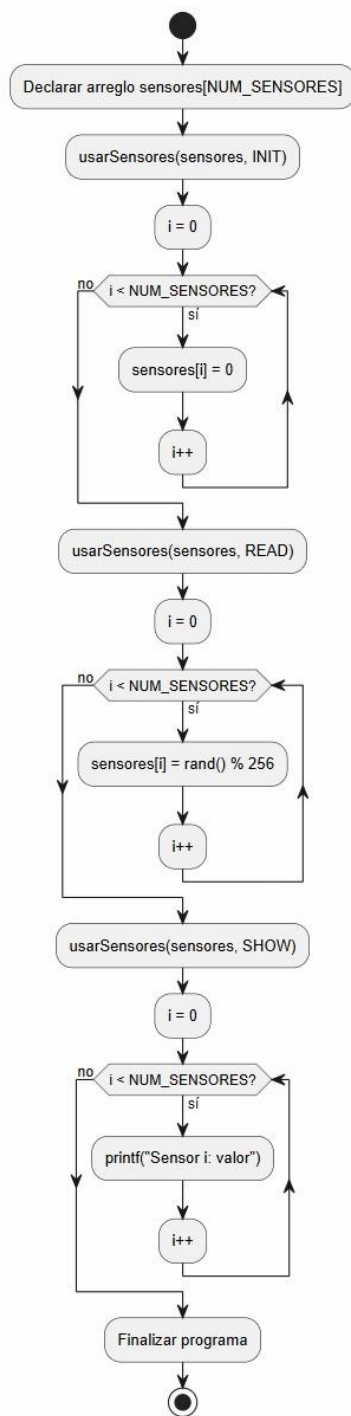


Figura 2. Diagrama de flujo la solución modular.

4. Parte 2: Medición de tiempos de ejecución

4.1. Objetivo

Realizar mediciones de tiempo de ejecución en un código sencillo con el fin de evidenciar las diferencias entre usar memoria del sistema y realizar operaciones de Entrada/Salida, como imprimir caracteres en pantalla.

4.2. Hito 1: análisis de la función `cicloEjemplo()`

La función `cicloEjemplo()` mide el tiempo promedio que demora un ciclo `for` en imprimir un asterisco en cada iteración. Para ello registra un tiempo inicial y final mediante `clock()`.

```
double cicloEjemplo(){
    int64_t cantidadIteraciones;
    double tiempoTotal;
    uint64_t inicio, fin;

    cantidadIteraciones = 1e6;
    tiempoTotal = 0.0;

    inicio = clock();
    for(int64_t i=0; i<cantidadIteraciones; i++){
        printf("*");
    }
    fin = clock();

    tiempoTotal = (((double)(fin - inicio)) / CLOCKS_PER_SEC) * 1e9;
    double tiempoPromedio = tiempoTotal / (double)cantidadIteraciones;
    return tiempoPromedio;
}
```

Concepto	Explicación
<code>clock()</code>	Retorna la cantidad de <u>ticks</u> de CPU consumidos por el programa desde que comenzó su ejecución.
<code>inicio y fin</code>	Almacenan <u>ticks</u> de CPU; no corresponden directamente a segundos.
<code>CLOCKS_PER_SEC</code>	Permite transformar los <u>ticks</u> a segundos, porque indica cuántos <u>ticks</u> equivalen a un segundo.
<code>(double)</code>	Convierte el resultado a número decimal de doble precisión y evita una división entera.

4.3. Hito 2: implementación de `cicloPrueba()`

Tomando como base `cicloEjemplo()`, se implementa `cicloPrueba(caso_t caso)`. La variable `caso` es un enum que permite seleccionar si el ciclo será vacío o si imprimirá un espacio, un asterisco o un salto de línea.

```
enum caso_t { VACIO, ESPACIO, ASTERISCO, NEWLINE };

inicio = clock();
switch(caso) {
    case VACIO:    for(int64_t i=0; i<cantidadIteraciones; i++) { ; } break;
    case ESPACIO:  for(int64_t i=0; i<cantidadIteraciones; i++) printf(" "); break;
```

```

case ASTERISCO: for(int64_t i=0; i<cantidadIteraciones; i++) printf("*"); break;
case NEWLINE:   for(int64_t i=0; i<cantidadIteraciones; i++) printf("\n"); break;
}
fin = clock();

```

Caso	Operación dentro del ciclo
VACIO	Ejecuta un ciclo sin operación interna.
ESPACIO	Imprime un espacio " ".
ASTERISCO	Imprime un asterisco "*".
NEWLINE	Imprime un salto de línea "\n".

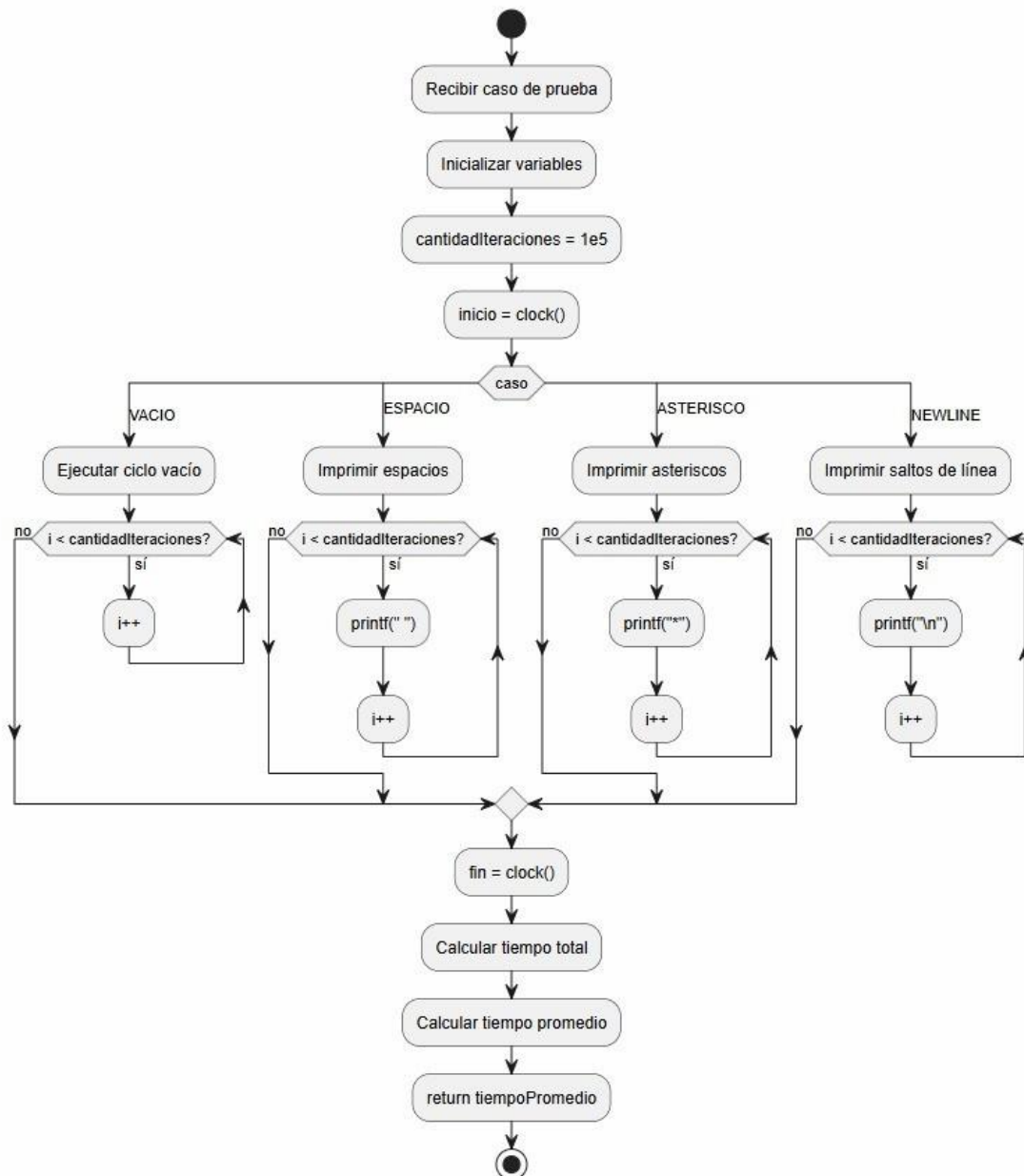


Figura 2. Diagrama de flujo de la función cicloPrueba(caso_t caso).

4.4. Hito 3: resultados

Para medir los tiempos de ejecución se utilizó la función `clock()`, proveniente de `<time.h>`. El método consiste en registrar el tiempo antes y después de ejecutar un bloque de código, en este caso un ciclo `for`. Ejemplo

```
inicio = clock();
for (int64_t i = 0; i < cantidadIteraciones; i++)
{
    printf("*");
}
fin = clock();
```

La diferencia entre `fin` e `inicio` representa la cantidad de ticks de CPU consumidos durante la ejecución del ciclo.

Para convertir este valor a segundos se divide por la constante `CLOCKS_PER_SEC`:

$t = \text{CLOCKS_PER_SEC} / (\text{fin} - \text{inicio})$

Luego el resultado se multiplica por 10^9 para expresar el tiempo en nanosegundos.

Una vez implementada la función `cicloPrueba()`, el programa se ejecutó 10 veces para cada caso de prueba. Los resultados se tabularon y se calcularon el promedio y la desviación estándar.

Caso	tprom1	tprom2	tprom3	tprom4	tprom5	tprom6	tprom7	tprom8	tprom9	tprom10	Promedio	Desv. Estándar
VACIO	10	10	0	9	11	10	10	9	10	11	9.00	3.16
ESPACIO	107980	66250	95820	88400	91300	84500	97800	90200	86100	93400	90175.00	11659.41
ASTERISCO	211760	65740	71950	74800	70600	68900	73500	72100	69900	75400	88365.00	43009.60
NEWLINE	91910	106130	93040	98200	101500	95700	103200	99900	97000	104800	99138.00	4916.80

Tabla 1. Resultados de tiempo promedio por caso de prueba en 10 ejecuciones.

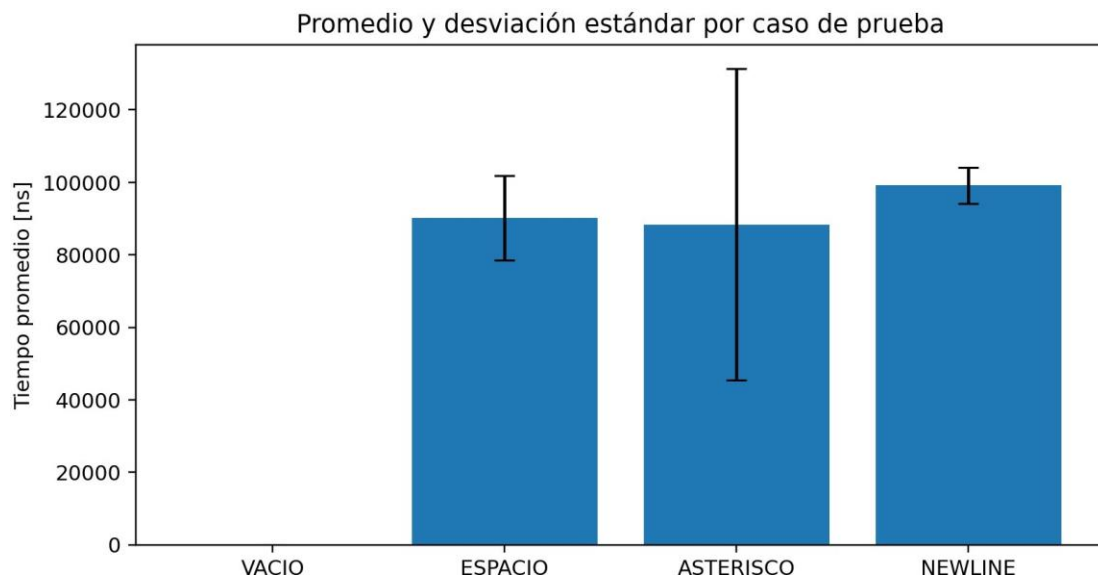


Figura 3. Promedio y desviación estándar de los tiempos obtenidos para cada caso de prueba.

Especificaciones Pc:

Procesador Intel Core i5-1135G7 a 2.4GHz

12gb RAM 3200 Mhz

4.5. Análisis de los resultados

El caso `VACIO` presenta el menor tiempo promedio, ya que el ciclo no realiza operaciones de Entrada/Salida. En

cambio, ESPACIO, ASTERISCO y NEWLINE presentan tiempos mayores porque utilizan `printf()` dentro del ciclo, lo que implica interacción con el sistema de salida.

El caso NEWLINE presenta el mayor tiempo promedio entre las mediciones principales, lo que puede explicarse porque imprimir un salto de línea puede involucrar un tratamiento adicional de salida. ASTERISCO presenta una desviación estándar alta, debido a que una de las mediciones fue mucho mayor que las demás, aumentando la dispersión de los datos.

5. Conclusiones

En la primera etapa de esta tarea, es posible evidenciar la existencia de la necesidad de transitar desde diseños monolíticos e individuales a arquitecturas estructuradas de manera más general, con la intención de optimizar la legibilidad del código y establecer un estándar de diseño robusto. Para ello, hemos compartimentalizado rutas de inicialización, lectura y visualización de sensores. Además, aplicamos principios de modularidad ya que nos resulta indispensable para el correcto desarrollo de sistemas escalables, garantizando un mejor rendimiento a la hora de integrar nuevo hardware futuro y sin comprometer la estabilidad de los procesos centrales.

Por otra parte, en la segunda parte de este entregable, enfocada en la medición de tiempos, se pudo constatar empíricamente las grandes diferencias en el costo de recursos computacionales de las diversas instrucciones gracias a la función `clock()`. Luego de evaluar los casos de prueba, se ha evidenciado que los ciclos que se operan de forma exclusiva en la propia memoria del sistema se ejecutan a velocidades óptimas. A diferencia de esto, al introducir operaciones de Entrada/Salida, como la impresión de caracteres y saltos de línea, se produce un gran y evidente cuello de botella. Todo esto se explica debido a las acciones como el vaciado de búfer terminaron por obligar al microprocesador a suspender su propio flujo de trabajo para sincronizarse con periféricos significativamente más lentos.

Con todo esto, comprobamos que, para programar rutinas de tiempo crítico, es preciso aislar y minimizar estas interacciones con el fin de optimizar y aprovechar realmente la verdadera capacidad de procesamiento del hardware a emplear